Table 6-3. *Reflection methods*

`getAttributes()`	Class attributes as `ReflectionAttribute` instances
`getConstants()`	Class constants as `ReflectionConstant` instances
`getInterfaces()`	Implemented interfaces as `ReflectionInterface` instances
`getMethods()`	Class methods as `ReflectionMethod` instances
`getModifiers()`	Class modifiers (e.g., `final`, `abstract`) as `integer`
`getName()`	Class name (including namespaces) as `string`
`getNamespaceName()`	Namespace(s) as `string`
`getShortName()`	Class name without namespaces
`getParentClass()`	Get `extends` class (if any) as `ReflectionClass` instance
`getProperties()`	Class properties as `ReflectionProperty` instances
`getTraits()`	Get traits used (if any) as `ReflectionTrait` instances

`hasConstant()`
`hasMethod()`
`hasProperty()`
	Test if member exists

`isEnum()`	`true` if object is an enum

The relevant constant/method/property reflection classes have similar methods available. For example, we've already seen `ReflectionProperty::getType()` used above to obtain a `ReflectionType` instance, on which we then called the `::getName()` method. This is the essence of the reflection API in PHP: language features have a corresponding reflection class, and those classes have logically named methods for inspecting the facets of those features. The `ReflectionMethod` class offers `::getReturnType()` and `::getParameters()`. `ReflectionEnum` class offers `::getCases()`, and so on… you get the idea. Add the following snippet to the previous code to see some of this in action:

```
print "... and methods" . PHP_EOL;
foreach ($refObj->getMethods() as $refMethod) {
    $modifiers = Reflection::getModifierNames($refMethod->getModifiers());
    $modifiers = implode(' ', $modifiers);
    print "  {$modifiers} {$refMethod->getName()}(): ";
```